

QUENTIN GUILLEREY

Customer Operations and Customer Success Manager

guillerey.finance@gmail.com | [linkedin.com/in/quentin-guillerey](https://www.linkedin.com/in/quentin-guillerey) | Columbia, MO

CSR AUTOMATION TOOLKIT

AI-Augmented Service Delivery Prototype | 2024–2025

Python · OpenAI API (optional) · Tkinter GUI · JSON response library · CSV audit log · Slack Webhooks (optional) · Google Sheets (optional)

STATUS: Working prototype, not yet at production scale. Architecture is fully functional end-to-end (classification, local audit logging, optional alerting/AI fallback). Currently covers 40 generic response entries as a proof of concept. Scaling to a full operational query taxonomy (the companion sanitized CSR query library runs to ~1,300 entries across 18 categories) is the active next phase. Code is public: github.com/Quentin-Guillerey/csr-automation-toolkit

THE PROBLEM

I was running U.S. operations for a Fortune 500 enterprise account handling 3,000+ customer contacts per month. The team had no standardized response system, no audit trail, and no mechanism to flag and route follow-up tasks. Agents spent most of their time on manual triage and copy-paste work, leaving little capacity for the interactions that actually moved the needle. No one was going to build the solution. So I started building one.

WHAT I BUILT

- **Tkinter GUI.** Desktop application for non-technical operators. No command line, no IT dependency. One-time setup screen (or "Skip" for fully offline use); the choice persists across launches.
- **Keyword classifier with optional AI fallback.** Rule-based matching as the primary engine, hardened with word-boundary and inflection handling so "sue" does not misfire inside "issue," with an optional OpenAI fallback for queries outside known patterns. Designed to extend to a full taxonomy.
- **Editable response library.** All responses live in an external JSON file, not in code. The app seeds an editable copy to each user's config folder on first run, so non-technical teammates can add or change responses in a text editor and reload in-app — no code changes, no rebuild. Invalid files fall back to the bundled default and log the error.
- **Local CSV audit log.** Primary logging method, zero setup required. Captures timestamp, query, response, follow-up flag, status, assignment, and resolution notes. Cells are escaped against spreadsheet formula injection, and any logging or alert failure is written to a visible error log rather than dropped silently.
- **Slack alerting (optional).** Every query triggers a notification if configured. High-priority items flagged [FOLLOW-UP NEEDED] for escalation.
- **Google Sheets logging (optional).** Writes a condensed row to a shared Sheet for team-wide visibility, if configured. Additive to the full CSV log, not a replacement. All network calls run off the UI thread to keep the app responsive.

NEXT STEPS

- Expand classifier coverage from 40 entries to a full, brand-agnostic query taxonomy
- Build a feedback loop so misclassified queries get flagged and folded back into the rule set
- Capture customer contact details in the GUI to populate the audit log's reserved contact field
- Package as a standalone executable for non-technical team rollout (no Python install required)
- Wire up Asana/Zapier auto-routing for flagged follow-ups
- Measure real performance once deployed at scale: time saved, escalation rate, audit coverage

WHY THIS MATTERS

This project is not a side experiment. It is what I do when I see a gap in a business I am responsible for: diagnose the problem, scope a practical solution, and build a working proof of concept without waiting for budget approval or a specialist. I built the architecture, put it in front of real use, and I'm scaling it deliberately rather than overstating where it stands today. The same instinct applies to any operations role: understand how a business's processes, data, and tools connect, identify where automation creates real leverage, and execute without over-engineering.